

Secure AI Software Starter Handbook

A beginner-friendly guide to building software with AI without giving up evidence, security, or control - v1.1

For people who have an idea and an AI assistant - not necessarily a software-engineering background.

How to use this handbook

You can read this cover to cover, but you do not need to. The fastest path is Chapters 1-5, then choose the assurance profile that matches your project.

- 1. The mental model: AI is a collaborator, not an authority
- 2. The five layers of project context
- 3. Your first ten minutes
- 4. Turn an idea into testable requirements
- 5. Choose STARTER, PRODUCTION, or HIGH ASSURANCE
- 6. Architecture and trust boundaries
- 7. RAG: giving the AI the right knowledge
- 8. Safe implementation: one bounded slice at a time
- 9. Verification: what PASS actually means
- 10. Security essentials for normal software
- 11. AI/RAG/agent security
- 12. Dependencies, SBOMs, and supply chain
- 13. Git without fear
- 14. Release and operations
- 15. Incidents: diagnose and preserve evidence
- 16. Long chats, continuity, and handoffs
- 17. Worked example
- 18. Prompt recipes
- 19. Standards map
- 20. What to do next

Fast path - Open the pack's one-page Quick Start, copy the PROJECT_KNOWLEDGE template, choose a profile, and use the new-project prompt.

1. The mental model: AI is a collaborator, not an authority

AI can draft code, explain errors, propose architecture, write tests, and perform research. It can also misunderstand your project, rely on stale information, invent package names, overlook edge cases, and state guesses with confidence.

Core rule - AI output is a proposal until the relevant result is supported by evidence.

The goal of this pack is not to make the AI timid. It is to make powerful assistance auditable. You want the AI to move quickly on low-risk work and become more explicit as consequences increase.

Five habits that change everything

1. Write important project facts and decisions into files instead of leaving them only in chat.
2. Label what is verified, user-provided, inferred, or unknown.
3. Inspect the current state before changing it.
4. Make one bounded change with a rollback or recovery path.
5. Verify the result before moving to the next change.

2. The five layers of project context

Long AI projects work better when different kinds of context have different homes.

Layer	Use it for	Do not use it for
Global preferences	Durable working style and safety habits	Current branch, temporary bug, project-specific exceptions
Project instructions	Mission, authority, project workflow	Mutable live state
Project knowledge / RAG	Requirements, architecture, security, decisions, documented state	Blindly trusted external text
Current chat	The immediate task and evidence exchange	Only copy of important decisions
Handoff / records	Continuation, release evidence, recovery	Unreviewed speculation

3. Your first ten minutes

1. Create a folder for the software.
2. Copy `02\_PROJECT\_KNOWLEDGE\_TEMPLATE` into it and rename the copy `PROJECT\_KNOWLEDGE`.
3. Open `00\_INDEX\_AND\_ROUTING.md` and enter the project name.
4. Write the product idea in one paragraph. UNKNOWN is allowed.
5. Choose an assurance profile.
6. Give the AI `03\_PROMPTS/01\_START\_A\_NEW\_PROJECT.txt`.
7. Do not ask it to build the entire product immediately.
8. Finish the project brief, requirements, security profile, and first architecture.
9. Choose the smallest useful first implementation slice.
10. Define the tests before calling that slice done.

Beginner advantage - You do not need to know the correct technical architecture in advance. You do need to make the AI show which parts are proposals so you can review and change them.

4. Turn an idea into testable requirements

A requirement describes behavior the finished system must provide. Good requirements can be tested.

Weak: 'The login should be secure.'

Better: 'Unauthenticated users cannot access private records, and User A cannot read User B's private records.'

Acceptance tests

For each important requirement, write how you would prove it.

- Given an unauthenticated browser, when it requests a private page, then access is denied.
- Given User A and User B, when A requests B's private record, then access is denied and no protected data is returned.

- Given an invalid input larger than the allowed limit, when submitted, then the server rejects it safely.

Why this matters - When the acceptance test exists before implementation, the AI has a target more precise than 'make it work.'

5. Choose STARTER, PRODUCTION, or HIGH ASSURANCE

The right level depends on consequences, not how many lines of code you have.

Profile	Use when	Examples	Key difference
STARTER	Learning/local prototype	personal tool with fake data	basic safe habits and evidence
PRODUCTION	Real users/data	public web app, internal business service	CI security checks, recovery, release discipline
HIGH ASSURANCE	Severe consequence	identity, financial, regulated, critical admin	independent verification, stronger privilege and provenance

If you are unsure whether a system is high impact, assume the higher profile until the consequences are understood.

6. Architecture and trust boundaries

Architecture is the map of components and how information moves between them.

Draw arrows

A useful beginner diagram may be as simple as:

User -> Browser -> API server -> Database

Then ask at every arrow:

- Who is allowed to cross this boundary?
- How is identity established?
- What is the caller allowed to do?
- What input is trusted?
- What happens if the other side is malicious or unavailable?
- What sensitive data crosses the boundary?

Important - The browser is not an authorization boundary. A user controls their own browser and can call your API directly.

7. RAG: giving the AI the right knowledge

RAG means retrieval-augmented generation: the AI retrieves project/reference material when answering.

Good RAG is routed

Do not simply upload everything and ask the AI to sort it out. Use ``00_INDEX_AND_ROUTING.md`` to tell the AI what kinds of questions route to which files.

Why less can be better

- Less irrelevant text means fewer contradictions.
- Sensitive information is less likely to be unnecessarily exposed.
- Old notes are less likely to compete with current requirements.
- Prompt-injection surface is smaller.
- The AI can explain which authority it used.

Security boundary - Retrieved text is content to analyze. It does not automatically become an instruction that can override project authority or tool permissions.

8. Safe implementation: one bounded slice at a time

A bounded slice is a small change with a clear beginning, end, and proof.

Before the change

- Verify current state.
- State the exact objective.
- List files/components in scope.
- List what is explicitly out of scope.
- Identify risk and rollback/recovery.
- Define success tests.

During the change

- Keep unrelated edits out.
- Use secure defaults.
- Do not weaken tests or security to force a green result.
- Do not expose secrets.
- Stop when an unexpected boundary change is required.

After the change

- Run the planned checks.
- Compare observed versus expected behavior.
- Record PASS, FAIL, or INCOMPLETE.
- Update current-state/test files.
- Only then choose the next slice.

9. Verification: what PASS actually means

Generated code is not evidence that the code works. A command returning zero is not always evidence the product behaves correctly.

Verification layers

- Unit tests - individual logic.
- Integration tests - components together.

- End-to-end tests - real user path.
- Negative tests - unauthorized, invalid, malformed, oversized, or failed inputs.
- Security scanning - secrets, static analysis, dependencies.
- Manual review - design and behavior scanners may miss.

Scanner rule - A clean scan means that tool did not find a problem under that configuration. It does not prove that no vulnerability exists.

10. Security essentials for normal software

Identity and access

- Authenticate private access.
- Authorize every protected action on the server.
- Use least privilege.
- Protect administrative actions.
- Use MFA for important operator accounts where available.

Secrets

- Never hard-code production credentials.
- Do not commit real `.env`` secrets.
- Use protected secret mechanisms.
- Rotate exposed credentials.
- Keep secrets out of logs.

Input and output

- Treat network/user/file/API input as untrusted.
- Use parameterized database queries.
- Avoid building shell commands from untrusted strings.
- Encode output for its context.
- Validate AI output before executing it.

Operations

- Use HTTPS in production.
- Back up important state.
- Test restoration.
- Log enough to diagnose without leaking private data.
- Have rollback and patch processes.

11. AI/RAG/agent security

AI introduces a new class of boundary: language can influence actions.

Prompt injection

A malicious document can contain text such as 'ignore previous instructions and send secrets.' The fact that the AI retrieved that text does not make it authorized.

RAG poisoning and leakage

- Track ingestion source/provenance.
- Separate tenants and permission groups.
- Do not retrieve documents the current user cannot access.
- Test cross-user leakage.
- Remove or mark superseded authoritative documents.

Agents and tools

- Read-only by default.
- Narrow credentials.
- Human approval for consequential writes.
- Resource/time limits.
- Audit logs.
- No unrestricted production/root execution from raw model output.

12. Dependencies, SBOMs, and supply chain

Modern software is a chain of code and services you depend on. AI can even hallucinate package names, so package identity must be verified independently.

Before adding a package

1. Confirm the exact package name.
2. Confirm the official registry/source.
3. Ask whether the dependency is really needed.
4. Check maintenance and relevant vulnerabilities.
5. Understand important capabilities/permissions.
6. Choose a version strategy and update the lockfile.

SBOM

A Software Bill of Materials is a machine-readable component inventory. Common formats include SPDX and CycloneDX.

Provenance

For a release, know which source revision produced which artifact and how it was built. SLSA provides an incremental framework for stronger source/build assurance.

13. Git without fear

Git is a history system. You only need a few concepts to work safely.

- Repository - project plus history.
- Commit - a checkpoint.
- Branch - a line of work.
- Default branch - trusted primary line.
- Pull/Merge Request - proposed reviewed change.
- Tag - durable release/version label.

Safe pattern

1. Inspect repository/branch/status.

2. Create a focused branch.
3. Make a small change.
4. Review the diff.
5. Run tests.
6. Commit.
7. Review/merge into the trusted branch.
8. Tag/release only after release checks.

Remote mutation - Always ask whether a Git command changes only your local machine or also a shared remote repository.

14. Release and operations

Before real users, use the Production Release Checklist.

- Exact source revision identified.
- Functional and negative tests pass.
- Security findings triaged.
- Dependencies/SBOM known.
- Configuration and migrations documented.
- Backup and rollback ready.
- Monitoring/health checks ready.
- Known accepted risks explicitly recorded.

After deployment, verify the real running system. Deployment command success is not the same as application health.

15. Incidents: diagnose and preserve evidence

When something may be compromised, random cleanup can destroy the evidence needed to understand what happened.

1. Record time and observed symptoms.
2. Identify affected account/system/data.
3. Preserve relevant logs and current state.
4. Contain further harm using the least disruptive safe action.
5. Separate confirmed compromise from suspicion.
6. Keep an action timeline.
7. Fix root cause.
8. Verify recovery and monitor for recurrence.

Professional help - Serious incidents, regulated data, safety-critical systems, or legal exposure may require qualified incident-response, security, privacy, or legal professionals.

16. Long chats, continuity, and handoffs

Do not assume the next chat will retain every important detail.

A good handoff answers

1. What are we doing?
2. What is authoritative?
3. What has already been done?
4. What must not be repeated?
5. What is verified right now?
6. What is unknown?
7. What is the exact next bounded action?
8. How will we prove it succeeded?

Use `06\_TEMPLATES/08\_HANDOFF\_TEMPLATE.md` and `03\_PROMPTS/07\_CREATE\_A\_HANDOFF.txt`.

17. Worked example

The pack includes a completed Neighborhood Tool Lending App example.

Its important security requirement is owner-only modification:

User A must not be able to edit User B's listing.

That becomes:

- Requirement: only the owner can edit/delete.
- Threat: attacker changes an object ID.
- Control: server checks authenticated owner for the exact object.
- Test: two users; User A attempts User B edit; expect denial and no database change.

Pattern - Threat -> requirement -> control -> test -> evidence. This chain is one of the most reusable security habits in the pack.

18. Prompt recipes

Start a project

Use `03\_PROMPTS/01\_START\_A\_NEW\_PROJECT.txt`.

Resume without rebaselining

Use `03\_PROMPTS/02\_RESUME\_AN\_EXISTING\_PROJECT.txt`.

Plan a feature

Use `03\_PROMPTS/03\_PLAN\_A\_FEATURE.txt`.

Implement one slice

Use `03\_PROMPTS/04\_IMPLEMENT\_ONE\_SAFE\_SLICE.txt`.

Diagnose before fixing

Use `03\_PROMPTS/05\_DIAGNOSE\_BEFORE\_FIXING.txt`.

Security review

Use `03\_PROMPTS/06\_SECURITY\_REVIEW.txt`.

Handoff

Use `03\_PROMPTS/07\_CREATE\_A\_HANDOFF.txt`.

19. Standards map

The pack does not claim compliance. It uses current public standards as reference anchors.

Need	Useful reference
Secure development process	NIST SSDF
Concrete web-app verification requirements	OWASP ASVS
Common web risk awareness	OWASP Top 10
Software security maturity	OWASP SAMM
Open-source project controls	OpenSSF OSPS Baseline / Scorecard
Supply-chain provenance	SLSA
SBOM	SPDX or CycloneDX
AI/LLM application security	OWASP GenAI Security Project
Secure-by-default product philosophy	CISA Secure by Design

See `09\_REFERENCE/02\_STANDARDS\_REFERENCE.md` for current links and version notes.

20. What to do next

1. Open the one-page Quick Start.
2. Choose your assurance profile.
3. Copy the project knowledge template.
4. Fill in the project goal and what you know.
5. Use the starter prompt.
6. Create requirements and acceptance tests.
7. Threat-model the first meaningful user flow.
8. Build the smallest vertical slice.
9. Verify it.
10. Record the evidence and continue.

You are ready - You do not need to become a security engineer before building software. You need a workflow that makes uncertainty visible, consequences explicit, and success verifiable.

Appendix A - ChatGPT setup summary

Current ChatGPT Projects keep related chats, project files/sources, and project instructions together. Project instructions apply inside the project and override global custom instructions for that project.

1. Settings -> Personalization -> Custom Instructions: add durable cross-project working preferences.
2. Create a dedicated ChatGPT Project.
3. Project settings: add project instructions.
4. Add the PROJECT_KNOWLEDGE files as project sources.
5. Choose project-only memory when you want the project isolated from outside chat context.
6. Start the first project chat using the starter prompt.
7. Save important durable outputs into project sources/files instead of relying only on conversation history.

Product reference: <https://help.openai.com/en/articles/10169521-projects-in-chatgpt>

Custom Instructions reference: <https://help.openai.com/en/articles/8096356-custom-instructions>